

Informe final del videojuego Dragon Baseball

Gustavo Valencia Mercado y Pablo Gonzalez Rojas

Universidad de Antioquia – Informática II
06 de Junio de 2026

Índice

1. Resumen	2
2. Objetivo y alcance	2
3. Requerimientos principales	2
4. Arquitectura del sistema	2
5. Diseño de niveles	4
5.1. Nivel 1: Vegito vs Freezer	4
5.2. Nivel 2: Gogeta vs Goku Black	4
6. Modelos físicos	5
7. Agente inteligente	6
8. Eficiencia y reutilización de recursos	6
8.1. Reutilización de proyectiles	6
8.2. Un objeto por tipo de ataque	6
8.3. Caché de imágenes	6
8.4. Menos actualizaciones innecesarias	7
9. Contenedores y memoria dinámica	7
10. Manejo de errores	7
11. Cambios frente a la propuesta inicial	7
12. Guía breve de uso	7
13. Conclusiones	8
14. Referencias	8

1. Resumen

Dragon Baseball es un videojuego en C++ con Qt que combina béisbol y combate en el universo de Dragon Ball. El proyecto tiene dos niveles: en el primero, Vegito batea proyectiles lanzados por Freezer; en el segundo, Gogeta enfrenta a Goku Black mediante movimiento lateral, saltos, esquives y ataques especiales.

Además de cumplir con la programación orientada a objetos, interfaz gráfica, físicas, dificultad y agente inteligente, se hizo énfasis en la eficiencia: los proyectiles se reutilizan mediante un *pool* fijo, las imágenes se cargan con caché y se reducen operaciones innecesarias durante el ciclo de actualización.

2. Objetivo y alcance

El objetivo fue construir un videojuego funcional, jugable y eficiente, usando C++ y Qt. El alcance incluye dos niveles diferenciados, tres o más modelos físicos, herencia propia, memoria dinámica, contenedores, sonidos, agente inteligente, manejo de excepciones y generación de ejecutable.

3. Requerimientos principales

Requisito	Estado	Evidencia
C++ y Qt	Cumple	Uso de <code>QMainWindow</code> , <code>QGraphicsScene</code> , <code>QTimer</code> , <code>QMediaPlayer</code> , señales y slots.
POO	Cumple	Clases <code>Personaje</code> , <code>Jugador</code> , <code>Villano</code> , <code>Proyectil</code> , <code>Sprite</code> , <code>Items</code> y <code>MainWindow</code> .
Herencia propia	Cumple	<code>Jugador</code> y <code>Villano</code> heredan de <code>Personaje</code> .
Dos niveles	Cumple	Nivel 1: Vegito vs Freezer. Nivel 2: Gogeta vs Goku Black.
Tres físicas o más	Cumple	Movimiento parabólico, oscilatorio, vuelo bateado y salto con gravedad.
Agente inteligente	Cumple	Implementado en <code>Villano</code> ; se reutiliza en Freezer y Goku Black.
Dificultad	Cumple	Modos fácil y difícil con variación de velocidad, daño y ataques.
Eficiencia	Cumple	Reutilización de proyectiles, caché de imágenes y menor cantidad de operaciones por frame.
Excepciones	Cumple	Validación de sprites, proyectiles y escenas con <code>std::runtime_error</code> .

4. Arquitectura del sistema

`MainWindow` actúa como controlador principal: administra la escena, niveles, teclado, HUD, sonidos, temporizadores y colisiones. Las entidades del juego se separan en clases para permitir una mayor organización, fácil entendimiento y debuggeado del código.

Clase	Responsabilidad
MainWindow	Coordina el flujo del juego, menús, timers, audio, colisiones y actualización de niveles.
Personaje	Clase base para entidades con vida, posición y animaciones.
Jugador	Acciones del usuario: bateo, control del proyectil y movimiento.
Villano	Enemigos y agente inteligente: percepción, razonamiento, acción y aprendizaje.
Proyectil	Estado, trayectoria, daño, reutilización y visualización de bolas o ráfagas.
Sprite	Carga y animación de hojas de sprites.
Items	HUD, textos, zonas de daño y elementos gráficos auxiliares.

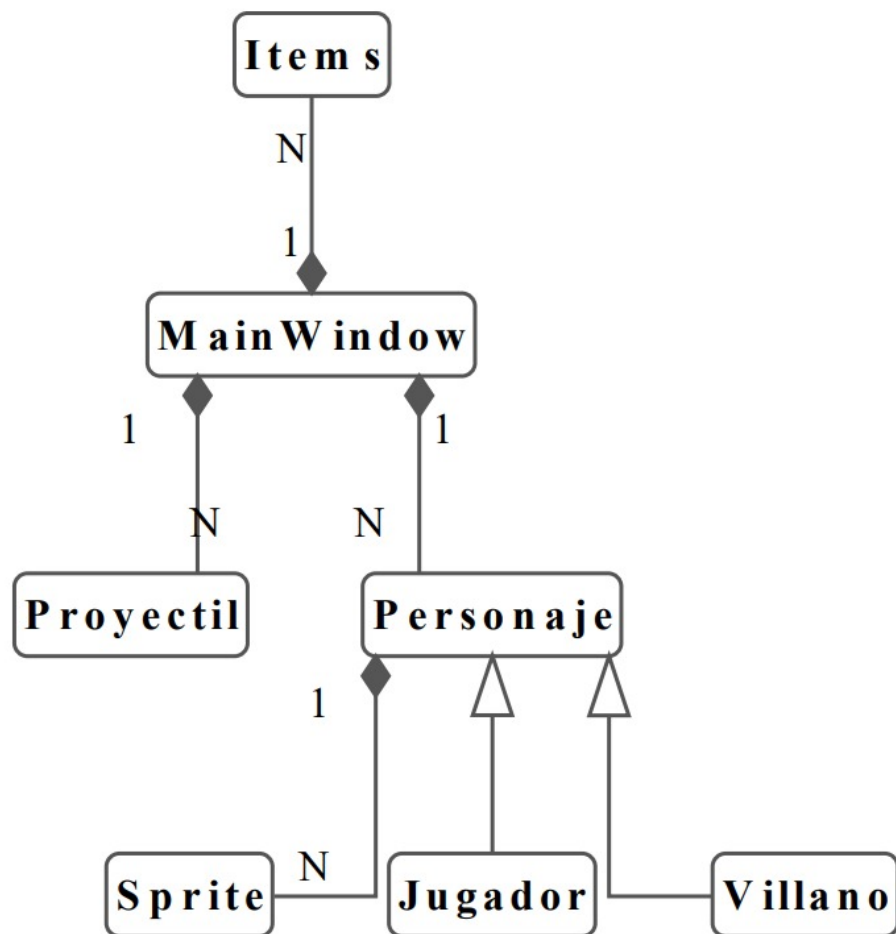


Figura 1: Diagrama de clases resumido

5. Diseño de niveles

5.1. Nivel 1: Vegito vs Freezer

El jugador controla a Vegito, quien debe batear los proyectiles de Freezer con la tecla P. Después del bateo puede corregir la caída con A, D, W y S. La bola cae sobre zonas de daño y reduce la vida de Freezer; si Vegito falla, pierde vida.

- Vista: cenital.
- Física principal: lanzamiento parabólico u oscilatorio y vuelo bateado.
- Dificultad: en modo difícil hay más tipos de ataque, velocidad y daño.

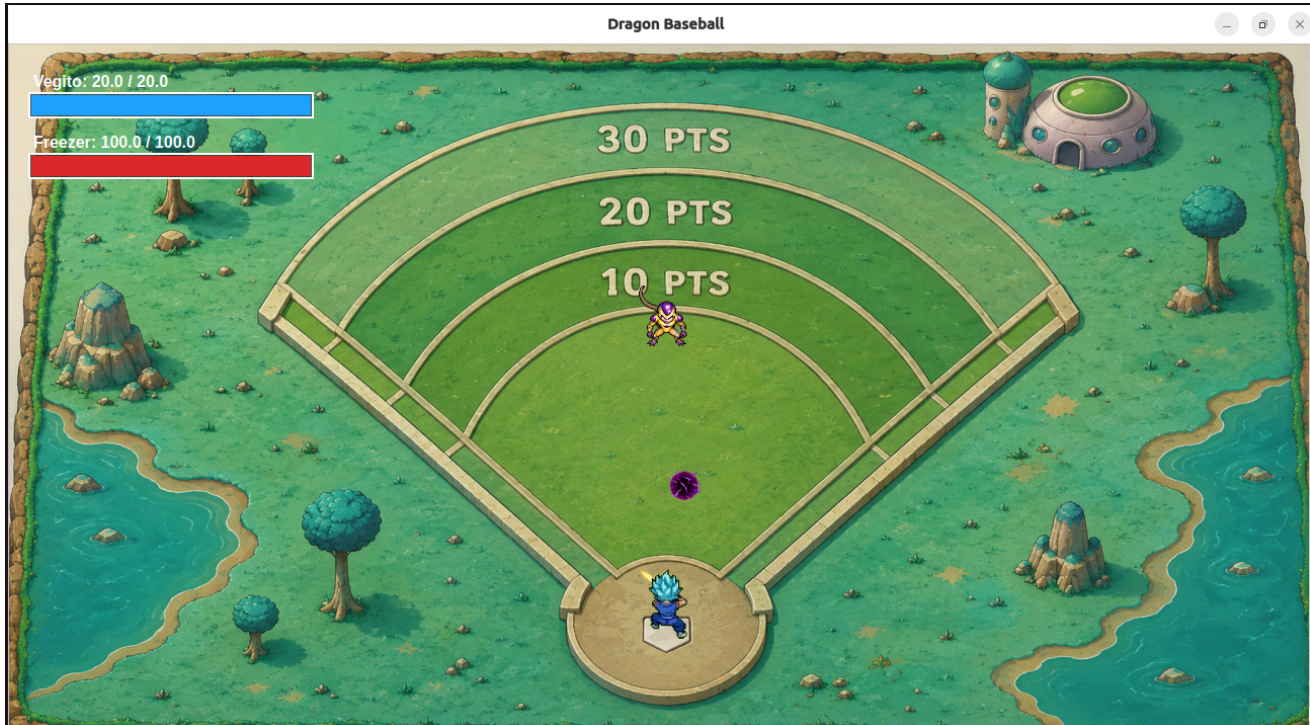


Figura 2: Vista del Nivel 1.

5.2. Nivel 2: Gogeta vs Goku Black

Gogeta se mueve y salta para esquivar o enfrentar ataques de Goku Black. En modo difícil se incorpora el Final Kamehameha como habilidad cargable. Black usa percepción de la posición del jugador para dirigir mejor sus ataques.

- Vista: combate lateral.
- Física principal: salto con gravedad y proyectiles dirigidos con movimiento parabólico.
- Dificultad: en modo difícil aparecen ataques especiales y daño directo por vida.

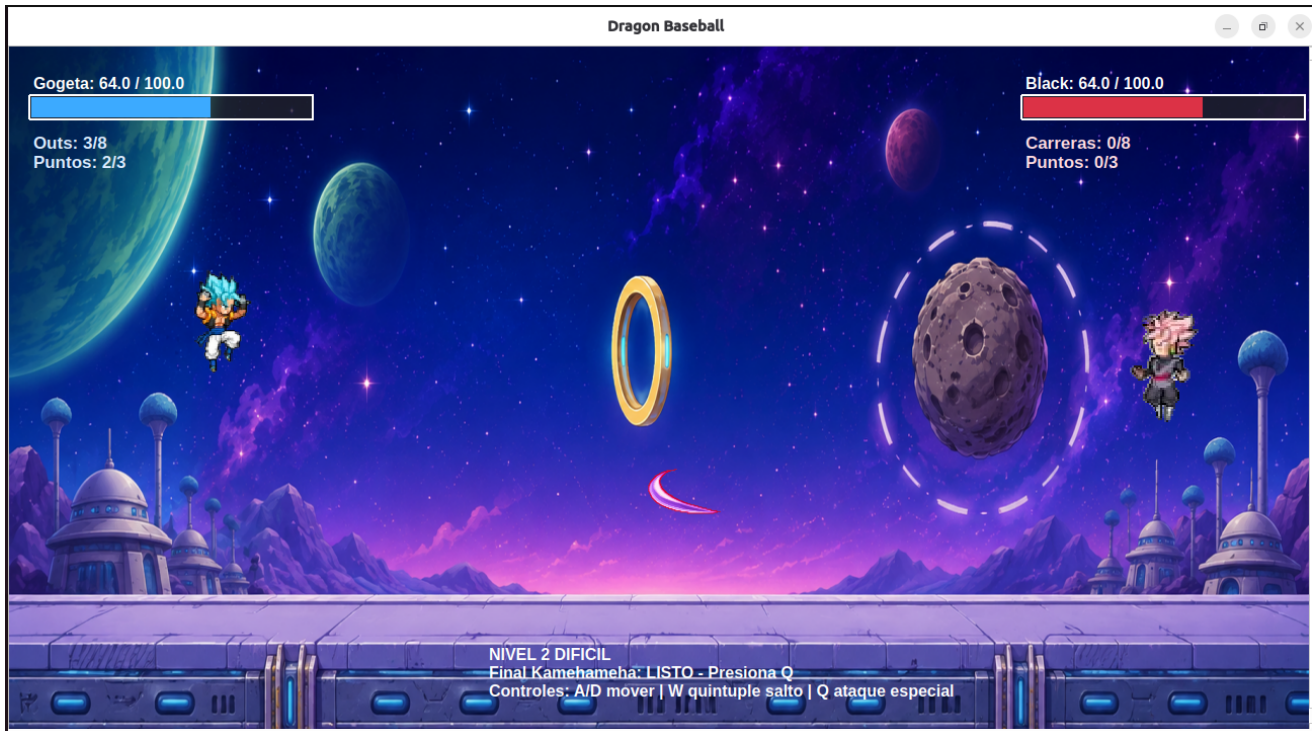


Figura 3: Vista del Nivel 2.

6. Modelos físicos

Modelo	Uso	Descripción
Movimiento parabólico	Lanzamientos de Freezer	La bola baja y se curva horizontalmente usando una variación suave.
Movimiento oscilatorio	Ataques difíciles	La bola desciende mientras oscila lateralmente, dificultando el bateo.
Vuelo bateado	Bola después del impacto	Se interpola hacia un destino con arco visual (escalando el tamaño de la imagen) y caída controlada.
Salto con gravedad	Movimiento de Gogeta	Velocidad vertical acumulada, gravedad y retorno al suelo.

Ejemplo conceptual del vuelo después del bateo:

$$y(t) = y_0 + (y_f - y_0)t - A \sin(\pi t)$$

El término senoidal simula la altura visual del batazo y evita que la bola se mueva como una línea recta.

7. Agente inteligente

El agente está integrado en **Villano**, no en una clase separada. Esto evita duplicar código y permite que Freezer y Goku Black compartan la misma estructura de decisión.

Componente	Método	Función
Percepción	<code>percibirPosicionJugador()</code>	Registra la posición del jugador.
Razonamiento	<code>razonar()</code> y <code>razonarDestinoLejano()</code>	Selecciona ataque o calcula un destino cercano al jugador.
Acción	<code>actuar()</code> / lanzamiento desde <code>MainWindow</code>	Ejecuta el ataque creando o reactivando un proyectil.
Aprendizaje	<code>aprender()</code>	Guarda aciertos y fallos por ataque para ajustar decisiones futuras.

Freezer lo usa principalmente para elegir ataques y aprender si el jugador batea o falla. Goku Black usa la parte más completa: percibe a Gogeta, razona un destino y lanza ataques dirigidos.

8. Eficiencia y reutilización de recursos

Esta fue una de las partes más importantes del proyecto. Se buscó reducir creación de objetos durante la partida, evitar cargas repetidas de imágenes y disminuir trabajo innecesario en cada actualización.

8.1. Reutilización de proyectiles

Se dejó una sola lista por enemigo: `bolasFreezer` y `bolasBlack`. Cada lista funciona como un *pool* fijo. La idea aplicada fue simple: se crea un proyectil por cada tipo de ataque. Cuando termina su uso, no se elimina ni se mueve a otra lista; se oculta con `desactivar()`. Cuando se vuelve a necesitar, se reactiva con `reiniciar()` desde una nueva posición.

```
if (!bola->estaActiva() && bola->getRutaSprite() == ataque.spriteProyectil) {  
    bola->reiniciar(posicionInicial, ataque.spriteProyectil);  
    bola->configurarAtaque(ataque.velocidad, ataque.dano);  
    return bola;  
}
```

Con esto se evitan tres costos: crear objetos repetidos, eliminar objetos durante la partida y mover punteros entre listas auxiliares.

8.2. Un objeto por tipo de ataque

Para Freezer se mantiene un proyectil reutilizable por cada sprite de ataque. Para Goku Black se reutilizan los proyectiles correspondientes a la bola normal y a la ráfaga especial. Si un proyectil de ese tipo ya está activo, no se crea otro igual, sino que simplemente se espera a que termine.

8.3. Caché de imágenes

Proyectil y Sprite usan `QHash<QString, QPixmap>` para no cargar ni escalar repetidamente las mismas imágenes. Esto reduce trabajo de CPU y evita repetir operaciones costosas como `QPixmap::scaled()`.

```
static QHash<QString, QPixmap> cacheProyectiles;  
if (cacheProyectiles.contains(ruta)) {  
    return cacheProyectiles.value(ruta);  
}
```

8.4. Menos actualizaciones innecesarias

También se redujo la actualización del HUD cuando no hay cambios de vida, puntaje o estado relevante. Esto evita refrescar textos y barras en cada frame sin necesidad.

9. Contenedores y memoria dinámica

Se usaron contenedores de Qt por compatibilidad con la arquitectura gráfica:

- `QList<Proyectil*>`: pool de proyectiles de Freezer y Black.
- `QList<Villano::Ataque>`: configuración de ataques.
- `QList<int>`: historial de aciertos y fallos del agente.
- `QVector<QRectF>`: recortes personalizados para animaciones.
- `QHash<QString,QPixmap>`: caché de sprites y proyectiles.

La memoria dinámica se mantiene para objetos dependientes de escena, como personajes, proyectiles, botones, audios y efectos. Para proyectiles, la política final es crear pocas instancias y reutilizarlas durante toda la partida.

10. Manejo de errores

Las excepciones se aplicaron solo donde son necesarias: carga de sprites, carga de proyectiles y validación de escena. Se usa `std::runtime_error` para no crear una clase adicional innecesaria.

```
if (pixmapOriginal.isNull()) {  
    throw std::runtime_error(  
        ("No se pudo cargar el sprite del proyectil: " + ruta).toString());  
}
```

`MainWindow` captura el error mediante `manejarErrorJuego()`, detiene temporizadores y muestra un mensaje controlado. Así el juego no continúa con recursos invisibles o inválidos.

11. Cambios frente a la propuesta inicial

- El agente quedó integrado en `Villano`, reutilizable para Freezer y Goku Black.
- La dificultad se concretó mediante selección de modo, velocidad, daño y variedad de ataques.
- Se modificó el diagrama de clases brevemente pensando en una mejor distribución.
- Se realizaron cambios menores en la lógica de juego inicial, pensando en algo más entretenido (más que todo en el nivel 2).
- Se cambiaron los personajes protagonistas.

12. Guía breve de uso

1. Abrir la carpeta con el ejecutable.
2. Ejecutar la aplicación.
3. Seleccionar nivel y dificultad.
4. En nivel 1, usar P para batear y A, D, W, S para corregir la caída.
5. En nivel 2, mover a Gogeta, saltar y usar el ataque especial cuando esté disponible.

13. Conclusiones

El proyecto integra POO, Qt, memoria dinámica, contenedores, físicas, dificultad, sonidos, manejo de errores y un agente inteligente sencillo. La mejora más relevante de la versión final fue la eficiencia: los proyectiles se reutilizan por tipo de ataque, las imágenes se almacenan en caché y se reducen actualizaciones innecesarias. Esto disminuye la creación de objetos durante la partida y hace que el juego sea más estable y defendible durante la sustentación.

14. Referencias

- Guía del Proyecto Final de Informática II, Universidad de Antioquia, 2026-1.
- Documentación oficial de Qt: <https://doc.qt.io/>
- Código fuente del proyecto desarrollado por el equipo.